2025-2026 CST1510 Programming For Data Communication and Networks

Module Lecturer: DR SUMITRA. KOTIPALLI

NAME: MICHAEL OLUWATOBILOBA ADEGORITE
MISIS:M01036168
PROGRAM: INFORMATION TECHNOLOGY
Title: MULTI-DOMAIN INTELIGENCE PLATFORM

Table of Content

INTRODUCTION

GitHub Repository: htttps://github.com/Michael-ade47/CST1510_CW2_Michael_Adegorite_O_M01036168.git

1 introduction

1.1 A multi-domain intelligence platform is an organized analytical system that integrates and unifies data from multiple domains; in our case, the cybersecurity, IT operations, and data science domains. It helps the user interpret scattered technical and operational data by turning it into clear, usable intelligence. Through the implementation of AI (artificial intelligence), the platform helps the user to analyze data, pinpoint issues, and identify trends amongst data through data visualization, and provide suggestions on how to correct identified problems or, if there are no major problems, provide insights on how to improve.

1.2 - 1.3 chosen domain and tier

Although I created all three domains—cybersecurity, IT operations, and data science—at Tier 3, in this document I will be focusing on the IT operations domain. The IT operations page is designed to manage and analyze IT support tickets related to IT issues, such as network faults. Each ticket has its own unique ticket ID, which is the primary key in the table, and is assigned to one of ten employees. The tickets are categorized by status (resolved, closed, open, or on hold), and each one has a priority (high, medium, low, or critical). Through this page, users can filter data to view the number of tickets with a certain status or priority. In addition, the system helps the user identify and view staff with the most delays that are holding back ticket resolution, including how many tickets they have that are delayed, as well as find the person with the highest number of delayed tickets. It also provides a domain-specific AI for IT to help aid with decision making. The page also includes CRUD functions, where the user can create and add new tickets, view tickets, update ticket status, and delete tickets, as well as hashing of users' passwords. This page also features OOP refactoring.

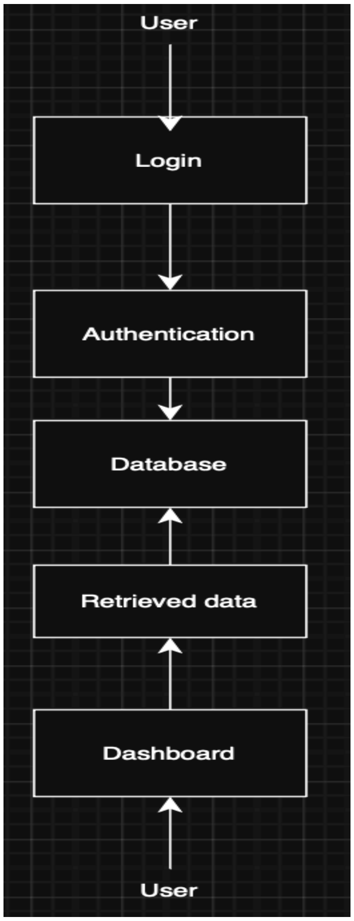
2.1 Security & Database

In the website, I have ensured security by using hashed passwords instead of plain text, as plain text passwords are highly vulnerable. If an attacker gains access to a device where passwords are stored in plain text, they can access to user accounts.In order to prevent this risk i use hashing. Unlike encryption, which can be decrypted (returned back to the original password), hashing is irreversible. Hashing is a mathematical algorithm that converts data into a fixed-length output known as a hash, and this process cannot be reversed. However, hashing on its own is not is not secure enough , as the same input will always produce the same hash as output. Due to this attackers can produce rainbow tables, which are tables containing commonly used passwords and their corresponding hashes. To prevent this, bcrypt is used to add what is known as a salt. A salt is a random value added to passwords before hashing to reinforce security.

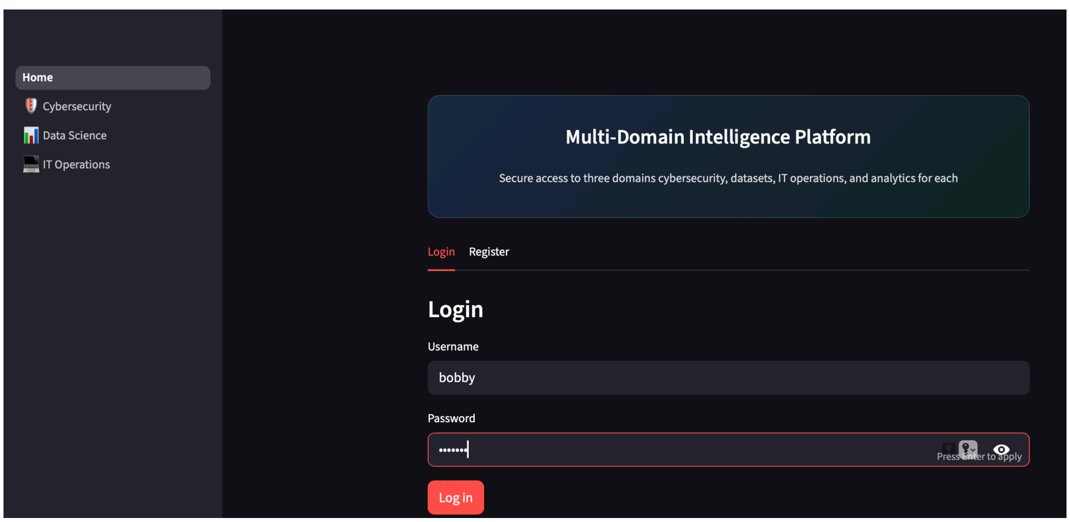
In addition to authentication securing, the system also enforces security at the database level.I use a structured relational database to store data in a secure manner. The data is organized into tables with a primary key, which enforces a unique constraint. In the IT_tickets table, the ticket ID is used as the primary key, as it is never repeated. The tables contain attributes and records, which allow us to query the data using SQL, as SQL enables filtering of data and the creation of relationships by linking tables through foreign keys. Security is reinforced by using parameterized queries(?, ?) to prevent SQL injection. SQL injection is a security vulnerability that happens when a system does not properly handle user input, allowing an attacker to insert harmful SQL commands. This can give the attacker unauthorized access to the database or allow them to though being unauthorized modify data or delete stored data. Also in order to mitigate SQL attacks I restrict user access and apply the principle of least privilege when granting database permissions.

1.2 System Structure

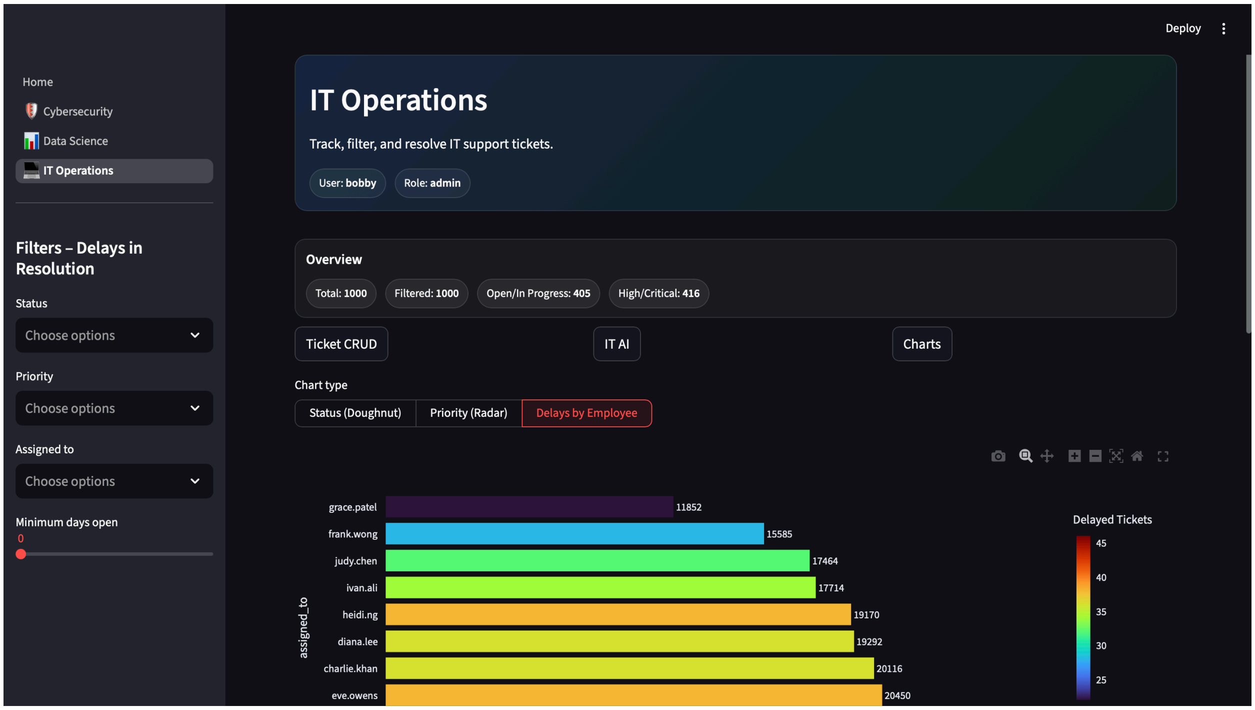
The diagram below displays how a user interacts with my streamlit page the top user represnets the input stage inputting his login details into the system while the user at the bottom represents the output stage where they receive output through the dashboard



Input stage :



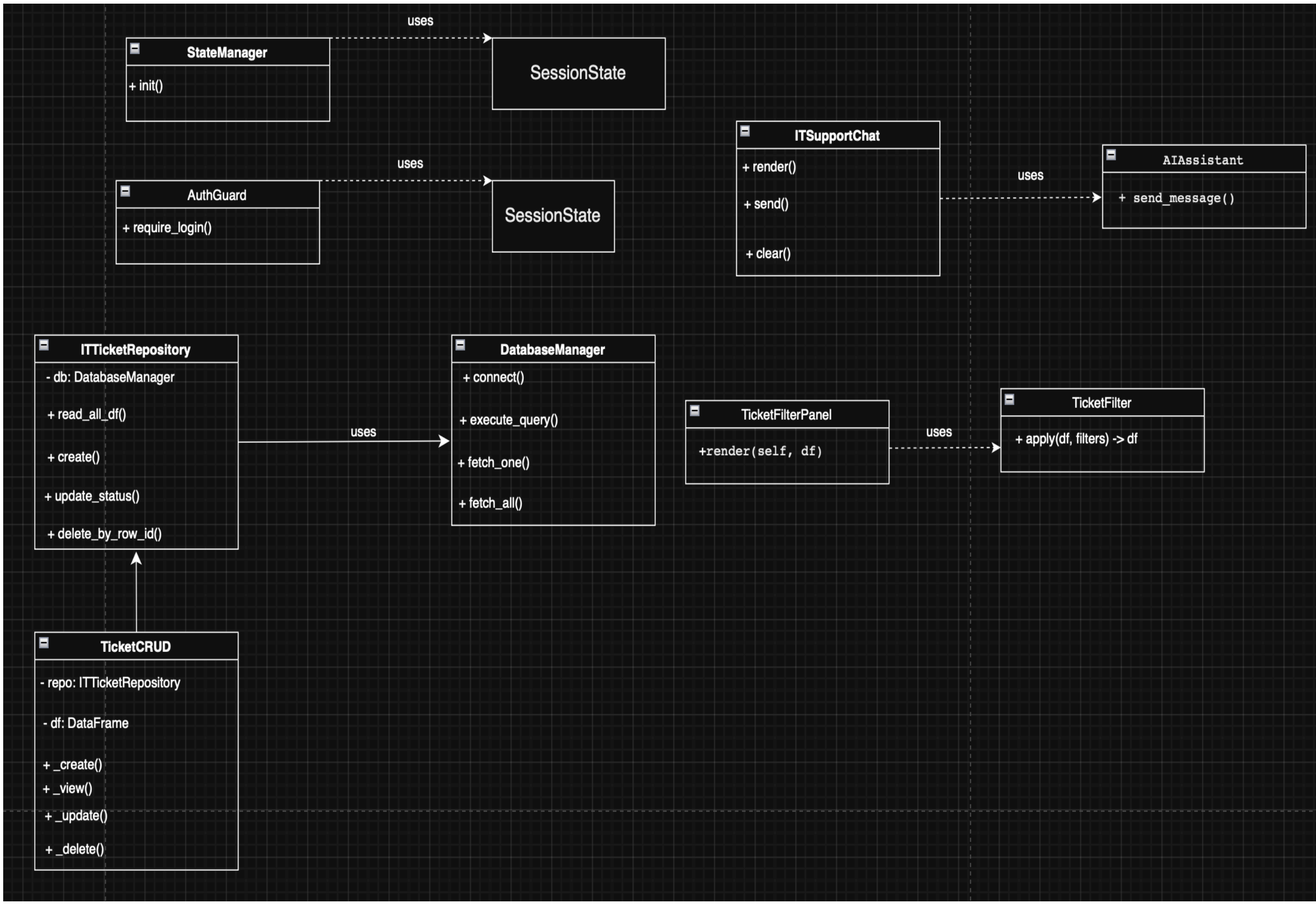
Output stage



The diagram shows how data flows within the my streamlit application. The user interacts with the login interface and it processed by the login logic.once the user is authenticated they are allowed access to the three domain dashboards. The application then retrieves the needed data from the database to then display as tables,charts and visualizations.

2.3 Code organization(oops refactoring)

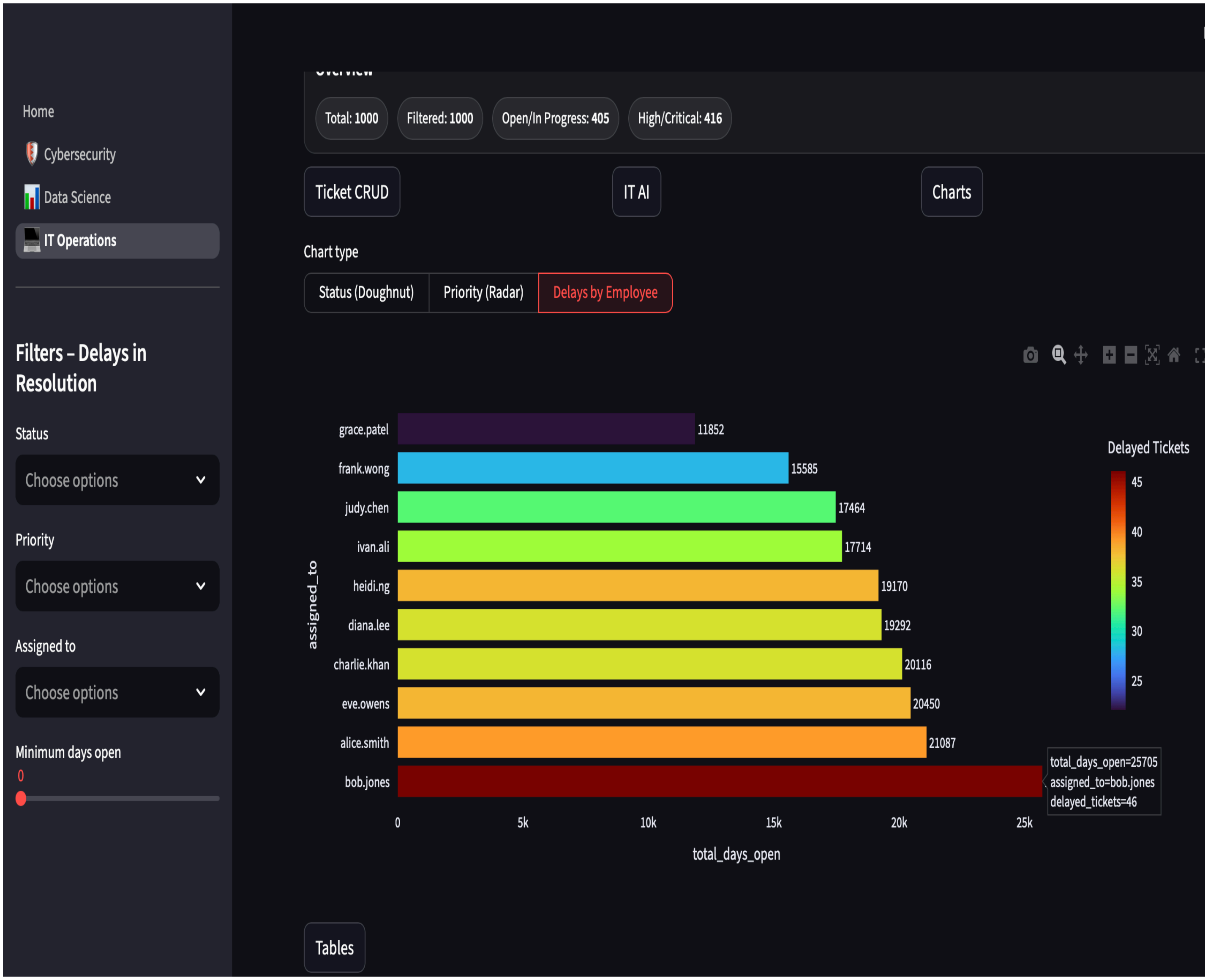
Unified Modeling Language(UML) diagram:-



My code was organized by using classes to split responsibilities and make the intelligence platform modular. DatabaseManager is a class responsible for establishing the connection with SQLite and, through the use of methods, reducing code lines, while the ITTicketRepository class is used to hold all CRUD operations for the IT operations page.

While on the user-interface side, I have a class called TicketsCRUD to manage the ticket CRUD forms and a TicketFilterPanel class that gets the filter selections from the sidebar and TicketFilter, which applies the filters to the charts. The StateManager class handles session control, and access is restricted by AuthGuard unless the user is logged in, while ITSupportChat deals with facilitating communication with AIAssistant to generate responses. This class-based structure reduces repetition and improves readability and simplifying code structure befor oop(object-oriented programming) my code lines for the IT operations page was over 500 words but now about 260.

2.4 Key Features



The image above is a horizontal bar chart that shows the user the delays by employee, from the employee with the lowest total delayed tickets at the top to the employee with the highest total delay in ticket resolution, so the user can know which employee has the greatest number of delayed tickets and those who are delaying ticket resolution, as well as the number of days the tickets have been open and how many tickets are delayed for that person. As we can see from the chart above, the employee with the highest number of delayed tickets is Bob Jones, with 46 delayed tickets.

Furthermore, the user can use the filters on the side to isolate data like the image below in which I have filtered to display the delay chart but with bob.jones as the assigned employee I can also filter by status or by priority.

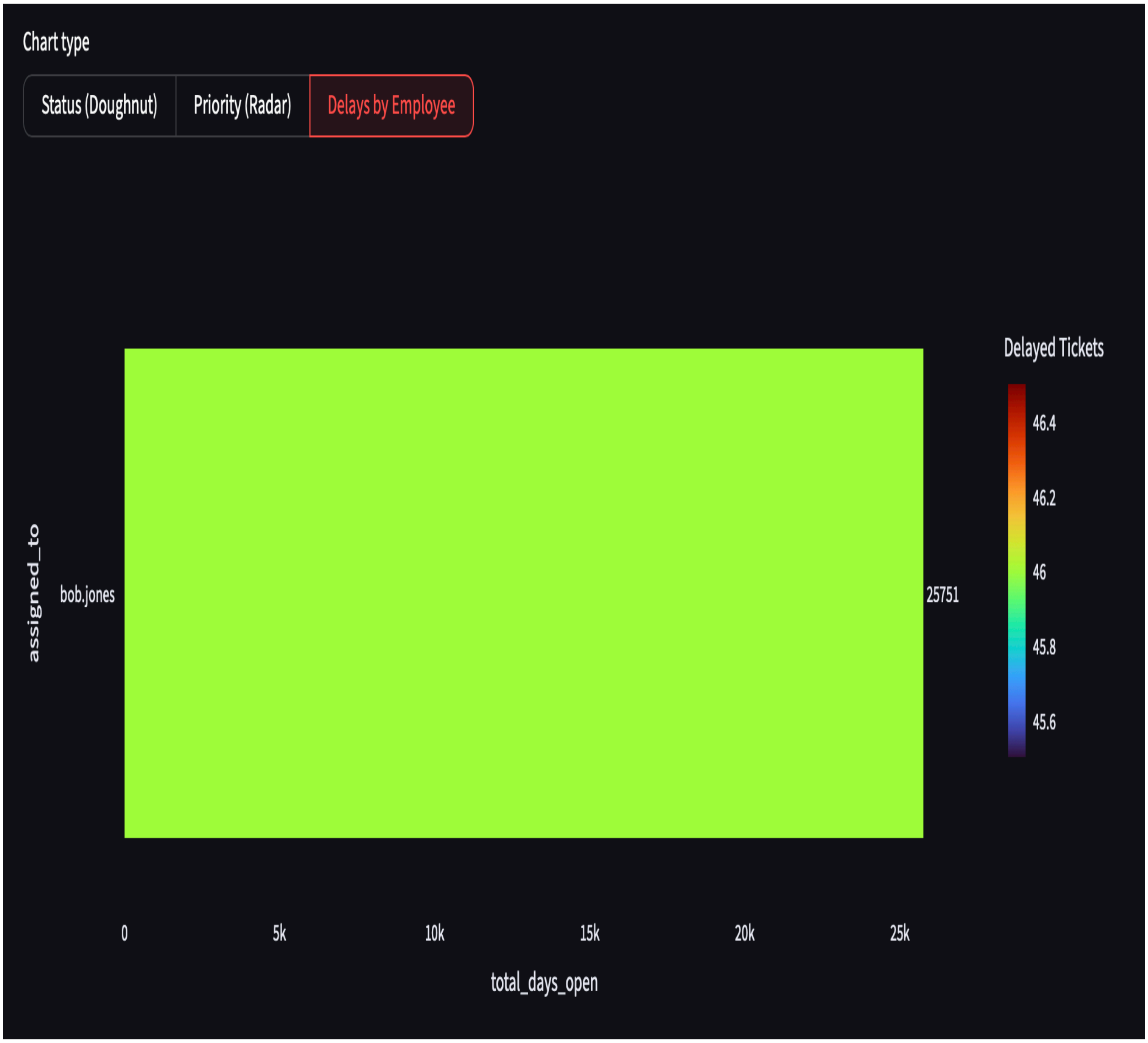
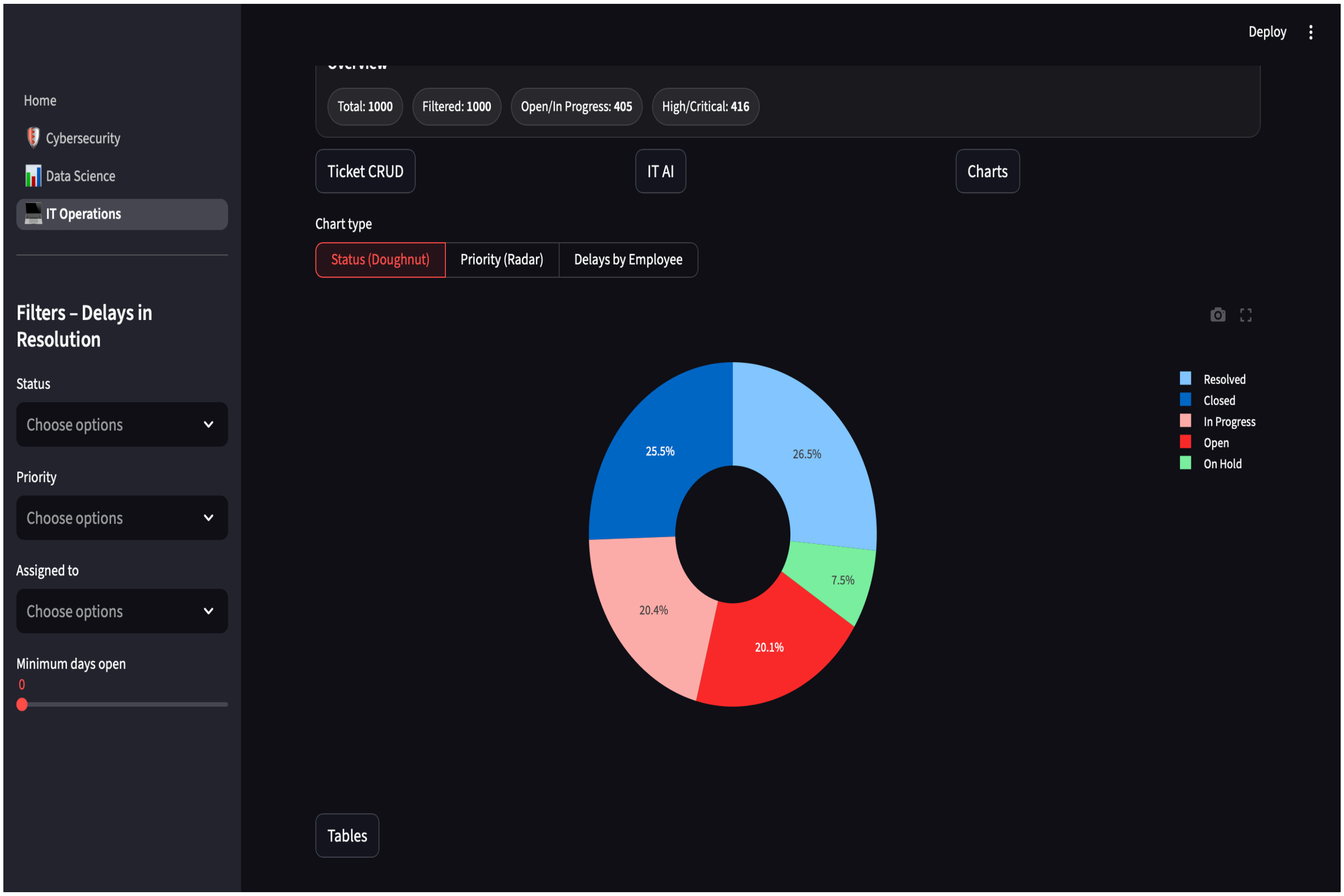
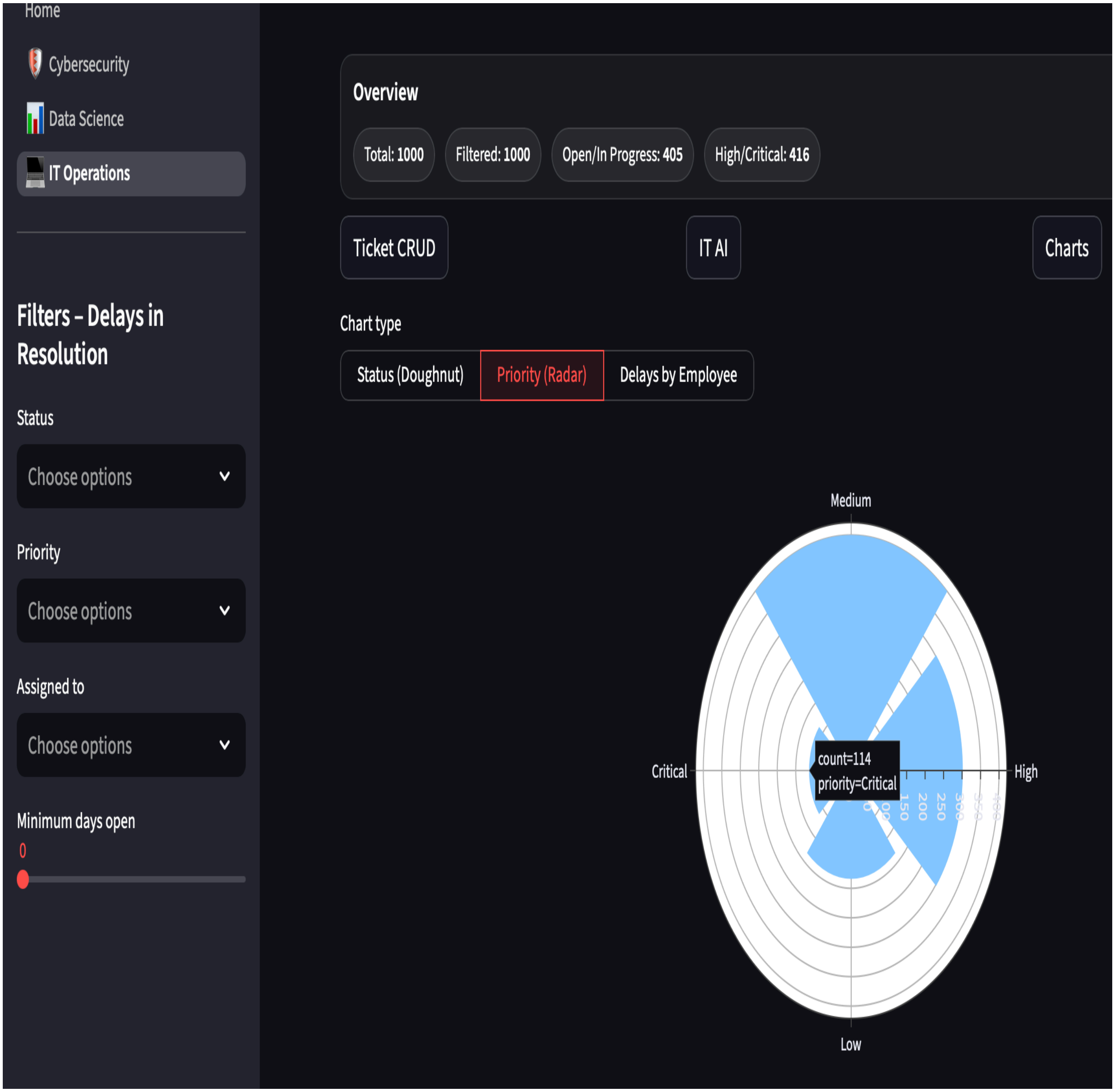


Chart type
Status (Doughnut)
Priority (Radar)
Delays by Employee
Delayed Tickets
46.4
46.2
46
45.8
45.6
assigned_to
bob.jones
25751
0
5k
10k
15k
20k
25k
total_days_open

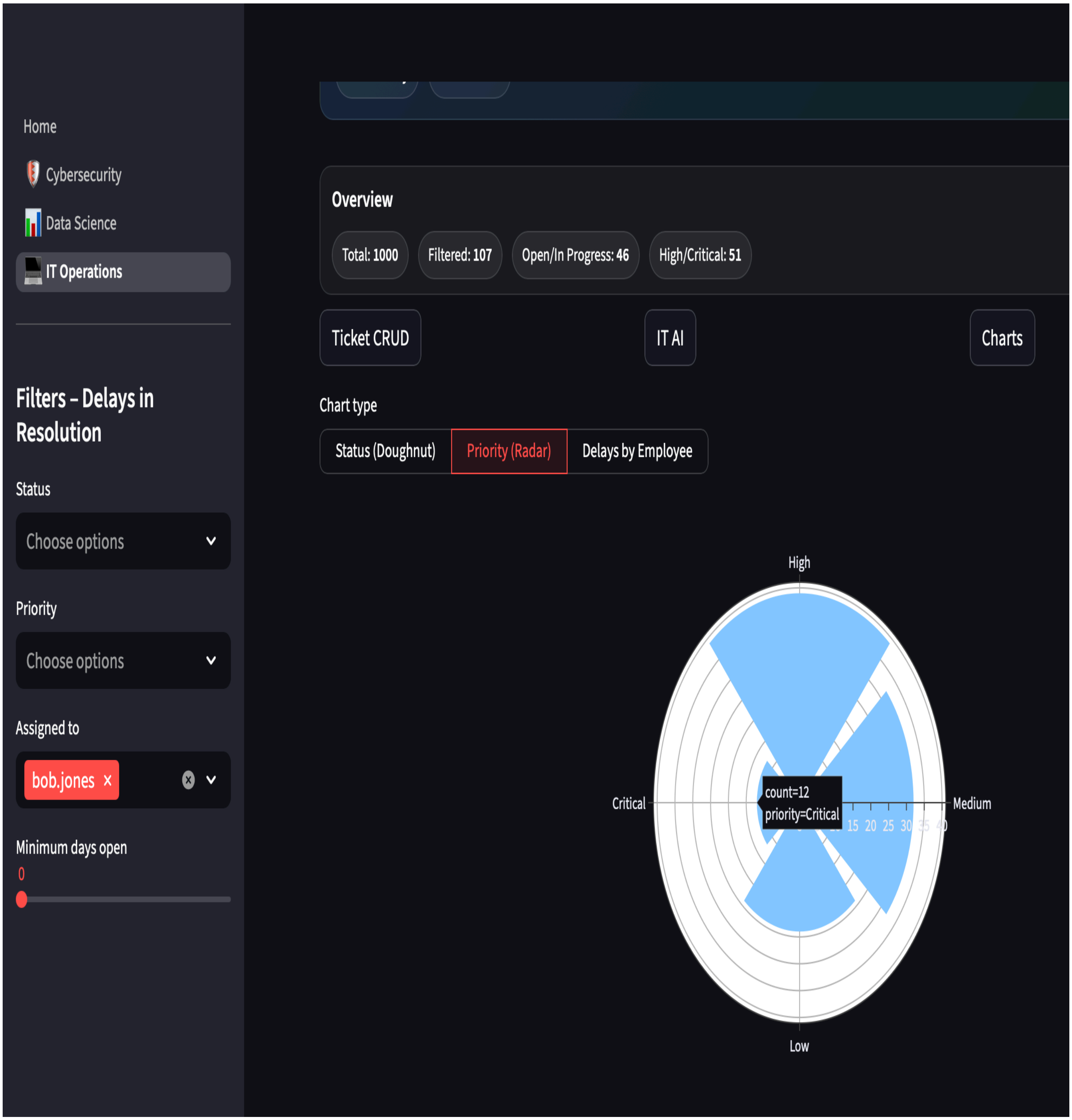
Below is a doughnut chart that displays the percentage distribution of IT tickets by status, allowing the user to easily compare the proportion of tickets in each status category. Through the sidebar filters I can find out how many tickets of which status each person has



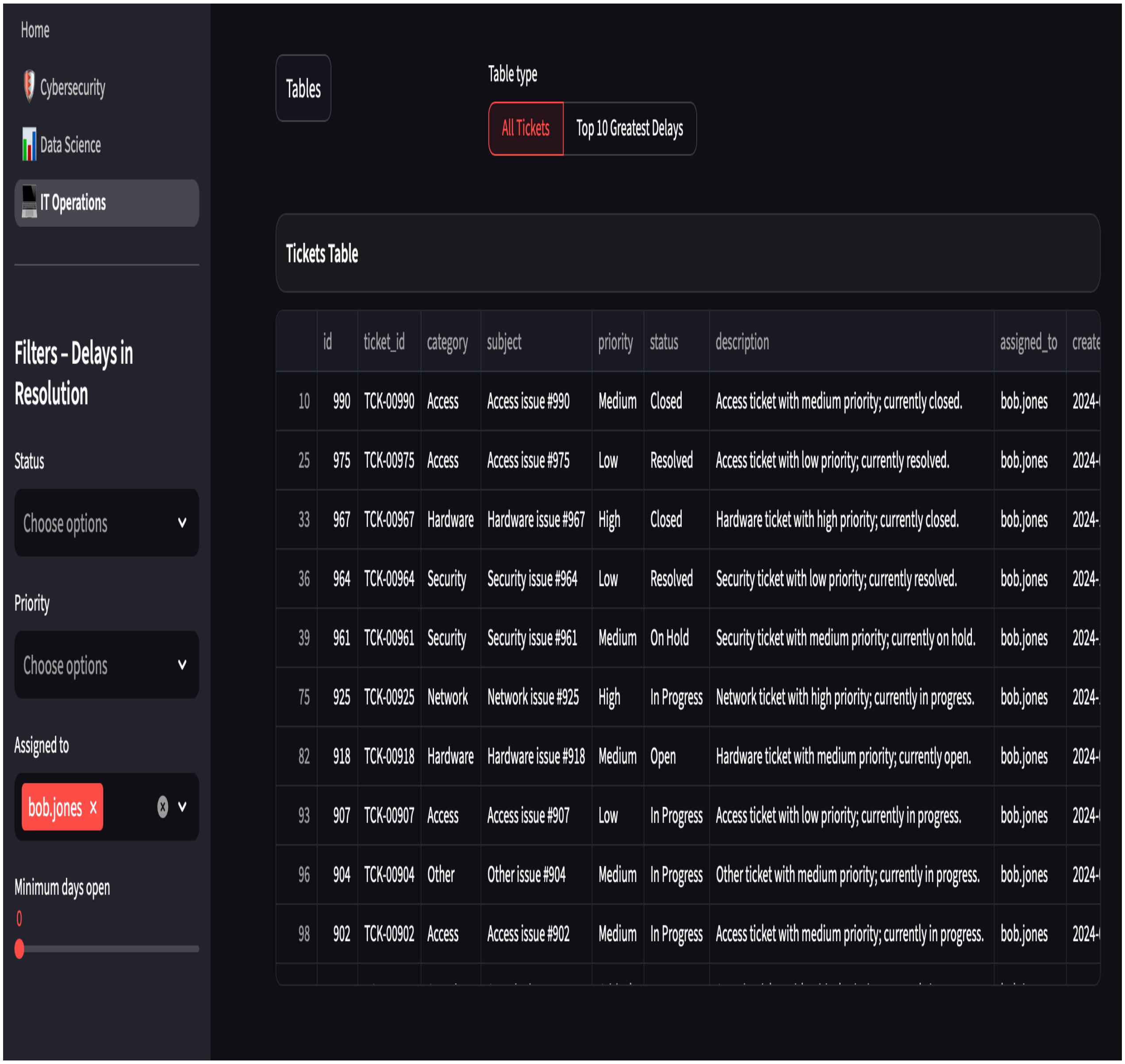
Below is a radar chart to display amount tickets of each priority from the digram beneath we can see the amount of tickets with priority critical is 114


Home
Cybersecurity
Data Science
IT Operations
Filters – Delays in Resolution
Status
Choose options
Priority
Choose options
Assigned to
Choose options
Minimum days open
0
Overview
Total: 1000
Filtered: 1000
Open/In Progress: 405
High/Critical: 416
Ticket CRUD
IT AI
Charts
Chart type
Status (Doughnut)
Priority (Radar)
Delays by Employee
Medium
Critical
High
Low
count=114
priority=Critical
150
200
250

when we filter assigned to for bob.jones. we find out he has 12 critical priority tickets



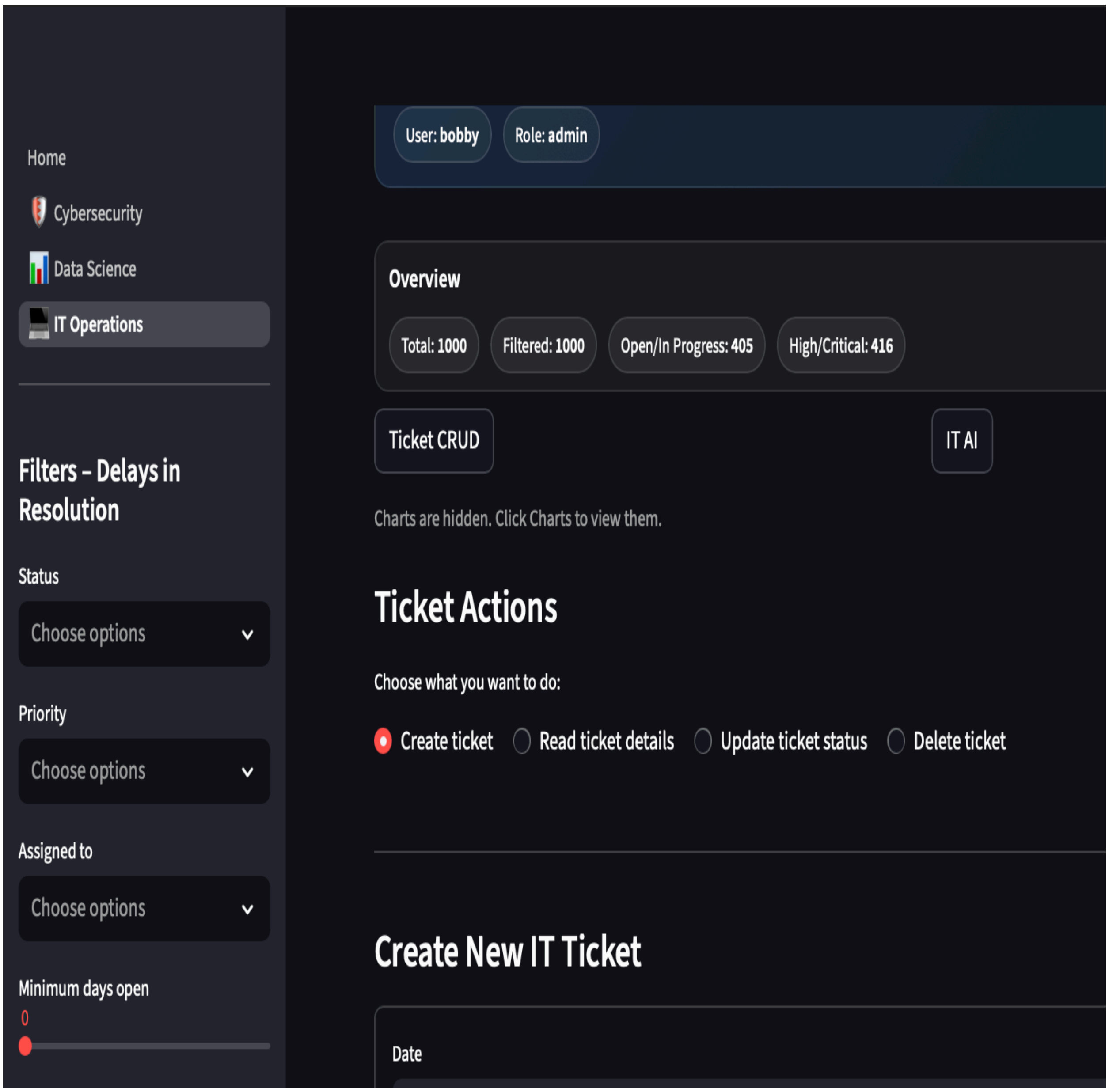
The filters also allow you to see all tickets assigned to which ever employee on the table e.g bob.jones



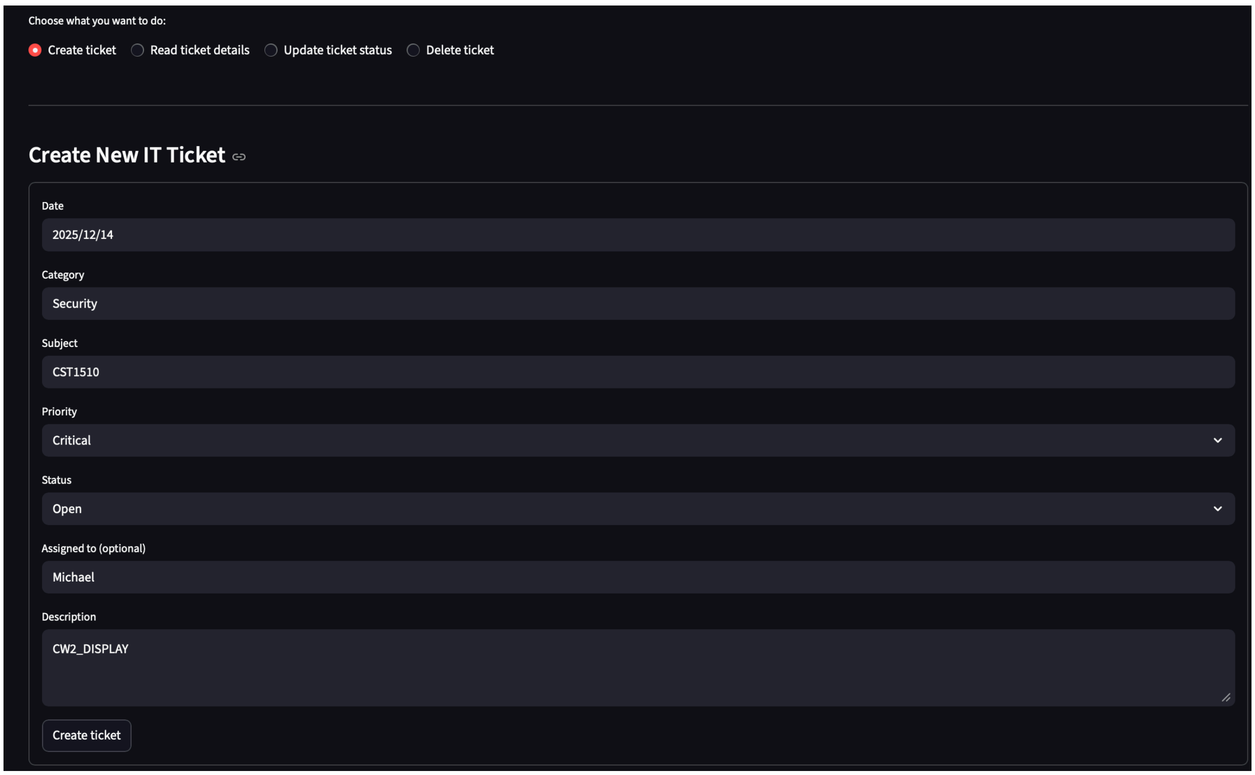
	id	ticket_id	category	subject	priority	status	description	assigned_to	create
10	990	TCK-00990	Access	Access issue #990	Medium	Closed	Access ticket with medium priority; currently closed.	bob.jones	2024-
25	975	TCK-00975	Access	Access issue #975	Low	Resolved	Access ticket with low priority; currently resolved.	bob.jones	2024-
33	967	TCK-00967	Hardware	Hardware issue #967	High	Closed	Hardware ticket with high priority; currently closed.	bob.jones	2024-
36	964	TCK-00964	Security	Security issue #964	Low	Resolved	Security ticket with low priority; currently resolved.	bob.jones	2024-
39	961	TCK-00961	Security	Security issue #961	Medium	On Hold	Security ticket with medium priority; currently on hold.	bob.jones	2024-
75	925	TCK-00925	Network	Network issue #925	High	In Progress	Network ticket with high priority; currently in progress.	bob.jones	2024-
82	918	TCK-00918	Hardware	Hardware issue #918	Medium	Open	Hardware ticket with medium priority; currently open.	bob.jones	2024-
93	907	TCK-00907	Access	Access issue #907	Low	In Progress	Access ticket with low priority; currently in progress.	bob.jones	2024-
96	904	TCK-00904	Other	Other issue #904	Medium	In Progress	Other ticket with medium priority; currently in progress.	bob.jones	2024-
98	902	TCK-00902	Access	Access issue #902	Medium	In Progress	Access ticket with medium priority; currently in progress.	bob.jones	2024-

CRUD FUNCTION BUTTON

In this page I have crud forms(Create new ticket, Read. Update status and deleteticket form it displays a button that allows you to hide or show the crud forms with a toggle button to select which form the user wants the forms and displayed)

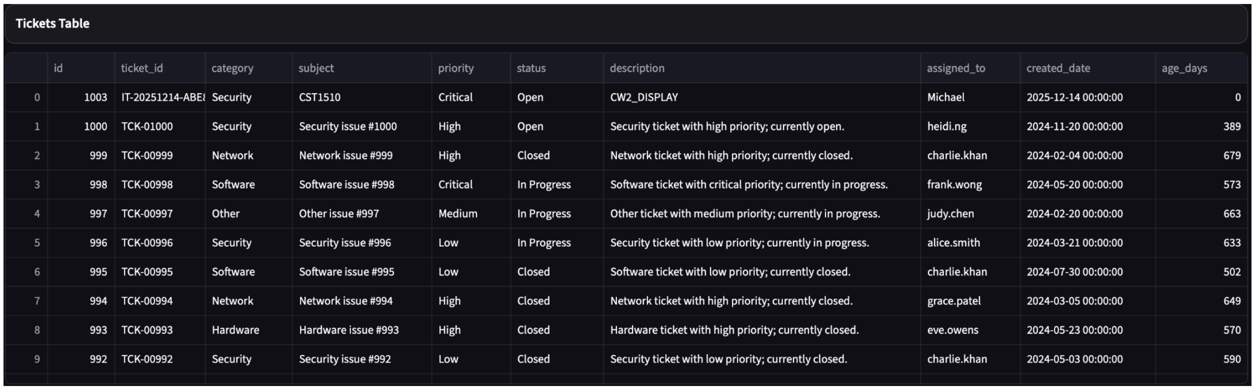


Create ticket form :-



Create ticket form output in table:-

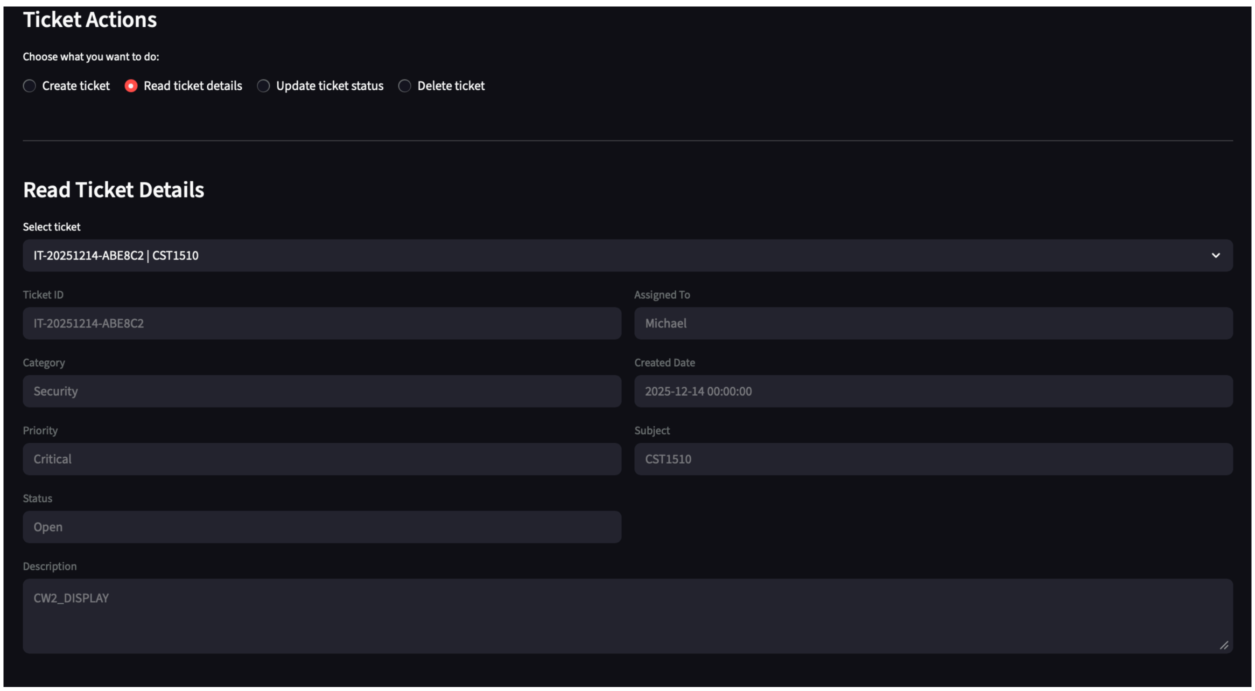
The ticket I recntly added is now at the top of the table



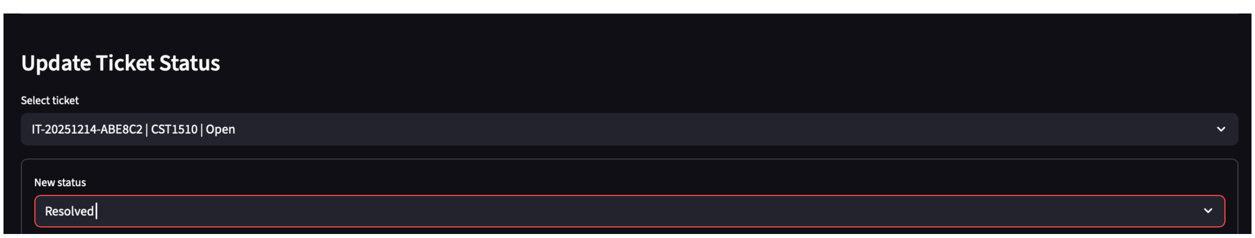
Tickets Table

	id	ticket_id	category	subject	priority	status	description	assigned_to	created_date	age_days
0	1003	IT-20251214-ABEI	Security	CST1510	Critical	Open	CW2_DISPLAY	Michael	2025-12-14 00:00:00	0
1	1000	TCK-01000	Security	Security issue #1000	High	Open	Security ticket with high priority; currently open.	heidi.ng	2024-11-20 00:00:00	389
2	999	TCK-00999	Network	Network issue #999	High	Closed	Network ticket with high priority; currently closed.	charlie.khan	2024-02-04 00:00:00	679
3	998	TCK-00998	Software	Software issue #998	Critical	In Progress	Software ticket with critical priority; currently in progress.	frank.wong	2024-05-20 00:00:00	573
4	997	TCK-00997	Other	Other issue #997	Medium	In Progress	Other ticket with medium priority; currently in progress.	judy.chen	2024-02-20 00:00:00	663
5	996	TCK-00996	Security	Security issue #996	Low	In Progress	Security ticket with low priority; currently in progress.	alice.smith	2024-03-21 00:00:00	633
6	995	TCK-00995	Software	Software issue #995	Low	Closed	Software ticket with low priority; currently closed.	charlie.khan	2024-07-30 00:00:00	502
7	994	TCK-00994	Network	Network issue #994	High	Closed	Network ticket with high priority; currently closed.	grace.patel	2024-03-05 00:00:00	649
8	993	TCK-00993	Hardware	Hardware issue #993	High	Closed	Hardware ticket with high priority; currently closed.	eve.owens	2024-05-23 00:00:00	570
9	992	TCK-00992	Security	Security issue #992	Low	Closed	Security ticket with low priority; currently closed.	charlie.khan	2024-05-03 00:00:00	590

Read ticket :-

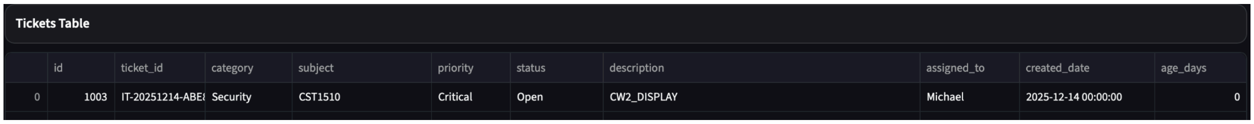


Update ticket status:-



Update ticket status output in table:-

Before:



Tickets Table

	id	ticket_id	category	subject	priority	status	description	assigned_to	created_date	age_days
0	1003	IT-20251214-ABEI	Security	CST1510	Critical	Open	CW2_DISPLAY	Michael	2025-12-14 00:00:00	0

After:

	id	ticket_id	category	subject	priority	status	description	assigned_to	created_date	age_days
0	1003	IT-20251214-ABEI	Security	CST1510	Critical	Resolved	CW2_DISPLAY	Michael	2025-12-14 00:00:00	0

Delete ticket:-

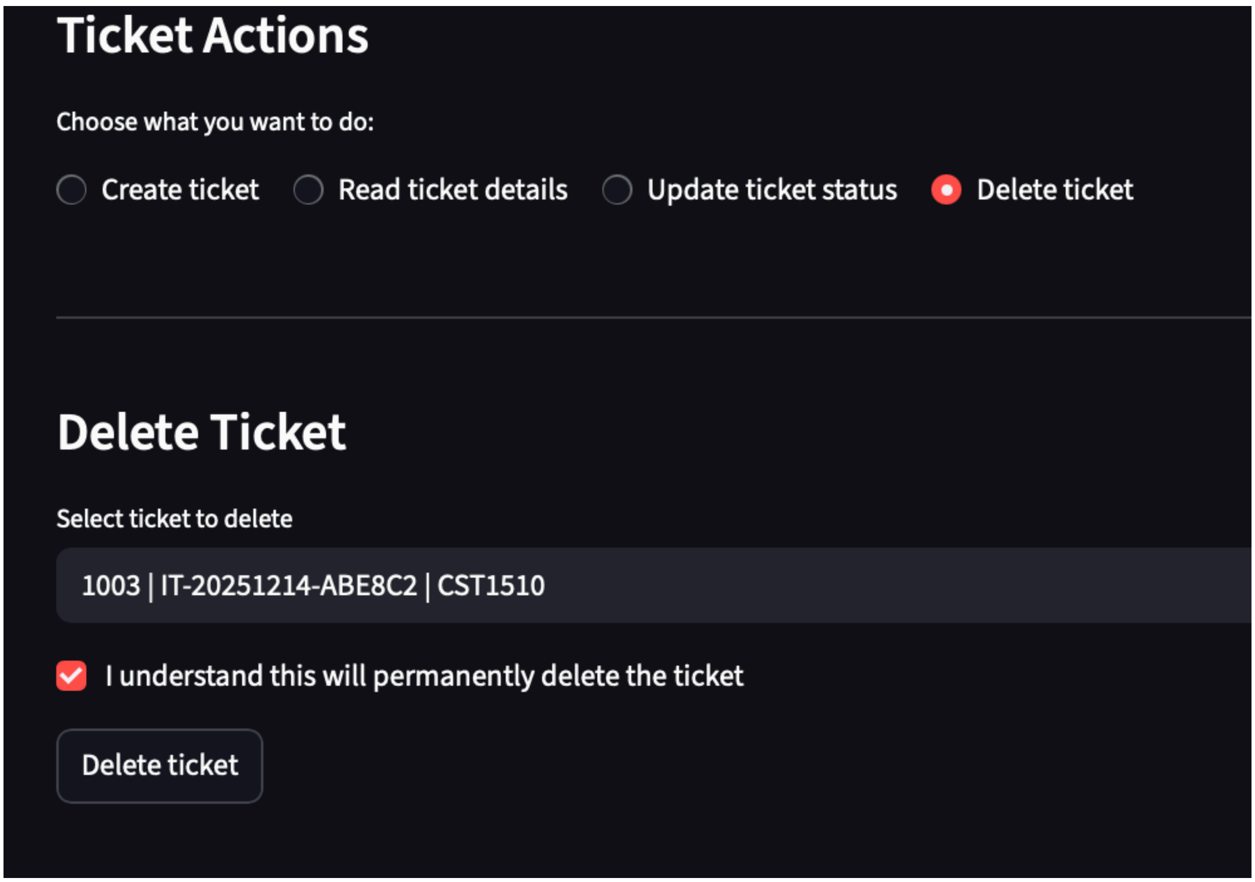


Table after deleting ticket:

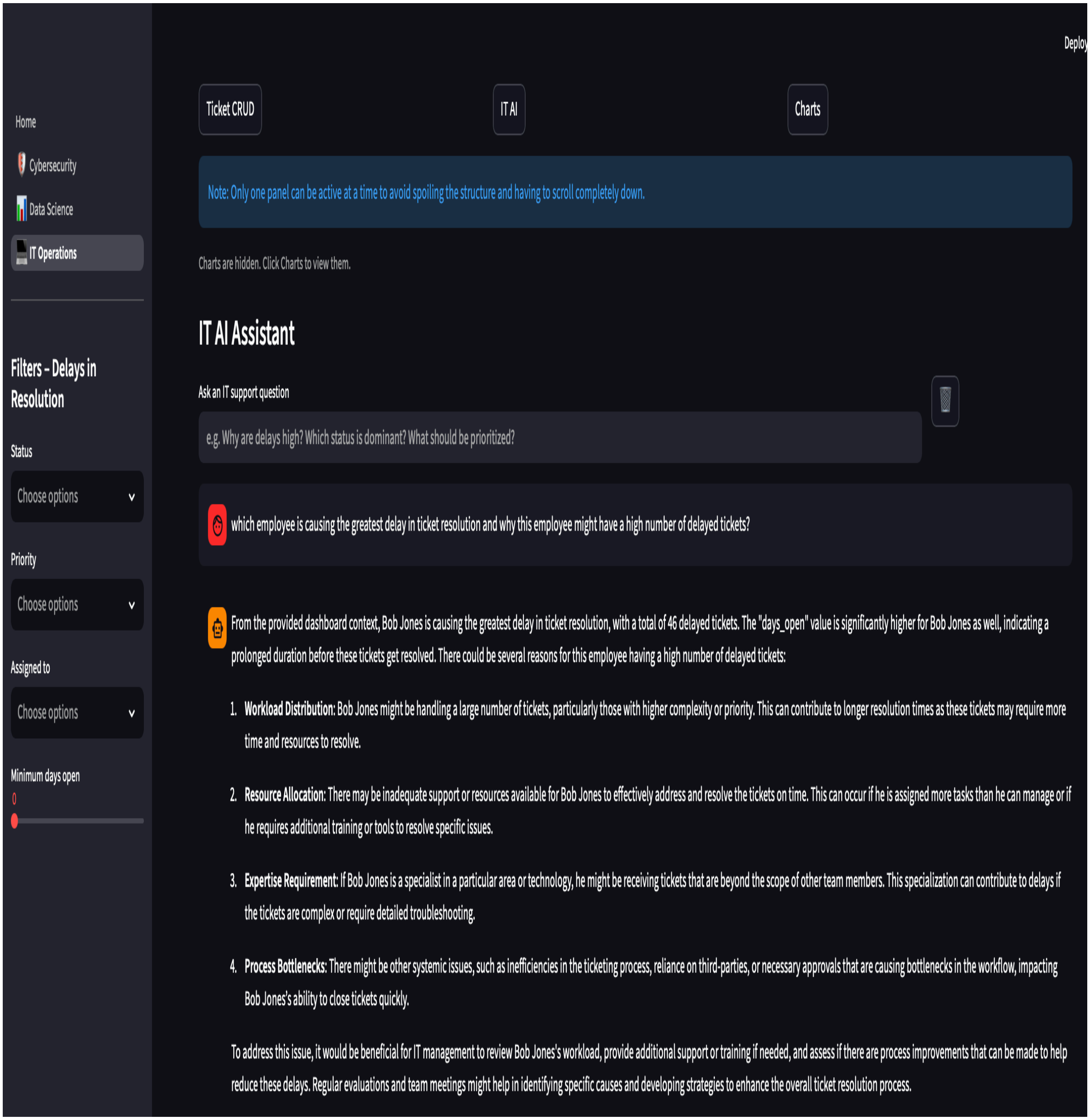
Tickets Table

	id	ticket_id	category	subject	priority	status	description	assigned_to	created_date	age_days
0	1000	TCK-01000	Security	Security issue #1000	High	Open	Security ticket with high priority; currently open.	heidi.ng	2024-11-20 00:00:00	389
1	999	TCK-00999	Network	Network issue #999	High	Closed	Network ticket with high priority; currently closed.	charlie.khan	2024-02-04 00:00:00	679
2	998	TCK-00998	Software	Software issue #998	Critical	In Progress	Software ticket with critical priority; currently in progress.	frank.wong	2024-05-20 00:00:00	573
3	997	TCK-00997	Other	Other issue #997	Medium	In Progress	Other ticket with medium priority; currently in progress.	judy.chen	2024-02-20 00:00:00	663
4	996	TCK-00996	Security	Security issue #996	Low	In Progress	Security ticket with low priority; currently in progress.	alice.smith	2024-03-21 00:00:00	633
5	995	TCK-00995	Software	Software issue #995	Low	Closed	Software ticket with low priority; currently closed.	charlie.khan	2024-07-30 00:00:00	502
6	994	TCK-00994	Network	Network issue #994	High	Closed	Network ticket with high priority; currently closed.	grace.patel	2024-03-05 00:00:00	649
7	993	TCK-00993	Hardware	Hardware issue #993	High	Closed	Hardware ticket with high priority; currently closed.	eve.owens	2024-05-23 00:00:00	570
8	992	TCK-00992	Security	Security issue #992	Low	Closed	Security ticket with low priority; currently closed.	charlie.khan	2024-05-03 00:00:00	590
9	991	TCK-00991	Access	Access issue #991	Medium	Resolved	Access ticket with medium priority; currently resolved.	alice.smith	2024-10-07 00:00:00	433

AI CHATBOT INTEGRATION

The IT domain features an AI assistant whose role is set to act as an IT expert. It is designed to relate its responses to the charts, tickets, and ticket statuses, and it operates through the use of an API key (Application Programming Interface) powered by OpenAI. The AI assistant serves as an analytical tool that provides insights into data trends, highlights potential issues, and suggests ways in which the user can resolve them. Through this, the user can detect likely causes of high numbers of delayed tickets and receive recommendations on how to reduce these delays, helping informed decision-making. It also allows the user see previous prompts aalong with their responses(it stores chat history using st.session_state).

The picture below shows the AI assistant response when I prompted it for which employee is causing the greatest delay in ticket resolution and why this employee might have a high number of delayed tickets?



It shows that Bob Jones is the employee with the greatest delay and the potential reasons why he has a high number of delayed tickets.

I prompted the assistant for a way I can reduce the amount of delayed tickets for Bob Jones and it provided. This as a response

how can we reduce the number of delayed tickets for Bob Jones

To reduce the number of delayed tickets for Bob Jones, several strategies can be implemented. Taking into consideration that Bob Jones has the highest number of delayed tickets at 46, here's a step-by-step approach:

1. **Analysis of Ticket Types:** Review the types of tickets Bob is handling. Identify if they fall under specific categories or involve particular types of issues or systems. This can help in prioritizing resources or providing specialized training where needed.
2. **Workload Assessment:** Evaluate the workload assigned to Bob compared to other team members. Ensure there is equitable distribution of tickets, especially those with high complexity or critical priority.
3. **Training and Development:** Provide Bob with additional training to bolster skills in areas where his tickets are delayed. This can enhance his efficiency and confidence in tackling complex issues.
4. **Resource Allocation:** Assign additional resources to help Bob with specific delayed tickets. This can include pairing with a team member for complex tickets or leveraging a knowledge base to resolve recurring issues more quickly.
5. **Process Improvement:** Look for any bottlenecks in the current workflow that Bob might be encountering, such as lengthy approval processes, dependencies on other departments, or inefficient tools. Streamlining these processes can significantly reduce resolution time.
6. **Regular Follow-Ups:** Implement a system of regular follow-ups on critical and high-priority tickets. Regular check-ins can help catch delays early and provide Bob with the support required to resolve tickets quicker.
7. **Use of Automation Tools:** Consider using automation to handle repetitive tasks such as ticket assignments, prioritization, and customer follow-ups. This can free up Bob's time to focus on more complex tickets.
8. **Feedback Loop:** Set up a system for feedback from Bob on why delays are happening. This can provide insights into specific barriers he faces and guide targeted improvements.

By applying these strategies, the number of delayed tickets for Bob Jones should decrease, improving overall efficiency and user satisfaction within the IT support framework. Regular evaluations and adjustments to these strategies will ensure continuous improvement.

3.1 Reflection

The most valuable skill I picked up is database management. In this I learned the advantage of using a database instead of file-based storage and how i can use queries to sort and filter data, unlike file-based storage, where i had to read the whole file line by line in order to query it, and how a database does not allow every user to modify the file, unlike file-based storage, which allows users to access and modify it concurrently, leading to file corruption, and with file-based storage, there are no backups. I also learned about SQL injection (a security vulnerability in which an attacker attacks SQL code by inserting malicious code into the user's input to modify or delete records). I learned that we can prevent this through the use of parameterized queries to make sure the user's input is treated as data, not code, so the attacker cannot insert any code in it.

Furthermore, to reinforce security and prevent SQL injection, we can restrict access to the database using the principle of least privilege (the user gets little access to data), in which we reduce the user permissions to mitigate damage from SQL injections.

In addition, we could link records in one table to another table using a foreign key (a constraint that is present in both tables) and use primary keys to make sure no rows are the same. A primary key is a unique identifier that is never repeated (it is also a unique constraint). Constraints are sets of rules that are applied to the database tables and also the use of queries such as "from… where age is less than 18" to filter the database. Overall, this helped reinforce the importance of security in web development.

Before this project, I underestimated how vulnerable databases were. I thought they were just more secure than file-based storage, not knowing they had their own risks as well as problems, but this has helped me realize that databases are not perfect or invulnerable until you use parameterized queries and the principle of least privilege to try and prevent SQL injections.

3.2 Challenges I faced

The main technical issues I faced were managing file paths and module imports due to how I structured my work. I was unable to locate certain modules due to relative path issues, and due to this, I would get import errors. Such as the diagram below

```
ImportError: cannot import name 'update_incident' from 'app.data.incidents'
(/Users/michaeladegorite/Projects/CST1510/app/data/incidents.py)

Traceback:

File "/Users/michaeladegorite/Projects/CST1510/my_app/pages/1_Dashboard.py", l
    from app.data.incidents import (
    ...<4 lines>...
    )

Copy  Ask Google  Ask ChatGPT
```

I solved this challenge by using Python’s pathlib module to find the project root and Python’s sys module to modify the system’s path alongside the use of __init__.py to treat the folder as a package to allow imports. This helped me understand Python package structure and how you must have an organized file structure when you want your work to run. For better accessibility and to reduce errors, as in an organized file structure, you can see where each file is clearly and underwhich folder they are under.

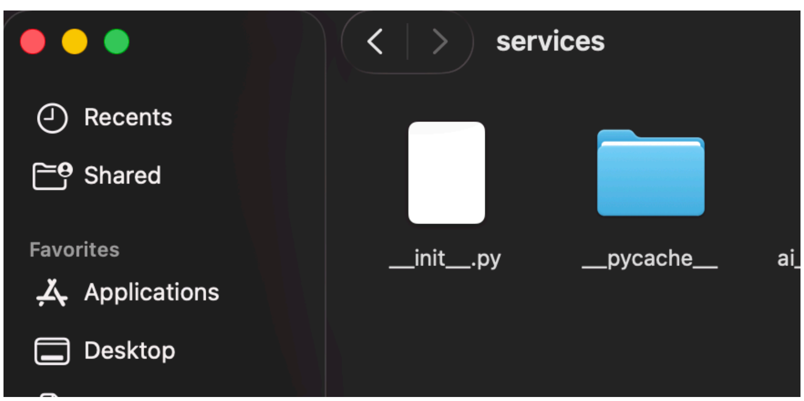
In the diagram below we can see I imported pathlib and sys

```
pages > 4_💻_IT_Operations.py > ...
import sys
from pathlib import Path
from datetime import date

import streamlit as st
import pandas as pd
import plotly.express as px

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))
```

As well as the use of __init__.py in the diagram below to make the services folder a package so i can import from it .



Conclusion

3.3 Conclusion

In this project I have been able to create Multi-Domain Intelligence platform that implements password hashing for authentication and salt as well as the use of SQL database for data management using parameterized queries as well as filtering to data and provide the user with visual representation of data through charts as well as tables. Also implementation of an AI assistant to help with analyze trends and provide insights on how to improve the process and simplify data from charts and tables in order to help in informed decision making .

Additionally, the integration of an AI assistant supports informed decision-making by analysing patterns and highlighting potential issues. Full CRUD functionality was implemented to allow effective management of tickets and incidents, and secure session handling ensures that user access and existing accounts are properly controlled. Overall, this project demonstrates a strong application of secure software development practices, object-oriented programming to increase scalability , and data-driven decision support.

In future projects, I would add role-based access control so normal users do not get the same permissions and access as admins do. This would reinforce security by not allowing normal users to perform things like deleting or modifying critical data, as well as audit logging to keep records of which user modified, deleted, or updated a certain record so as to provide accountability for the actions of users.

References

OWASP Foundation (2023) *SQL Injection*. Available at: https://owasp.org/www-community/attacks/SQL_Injection(Accessed: 12 December 2025).

OWASP Foundation (2023) *Password Storage Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (Accessed: 12 December 2025).

Oracle (2023) *Object-Oriented Programming Concepts*. Available at: https://docs.oracle.com/javase/tutorial/java/concepts/ (Accessed: 14 December 2025).

Lecture slides from week 7-11

AI Usage:

AI's used – Quilbot punctuation chekcer to correct punctuationa and grammar issues I would first write my paragraph then use quilbot's punctuation checker to fix punctuation and gramartical issue.

Chatgpt – csv generation ,ideas to implement.